"Enable Build Scale"

Platform Engineering Notes

Observability, Golden Paths & Self-Service Infrastructure

1. Introduction

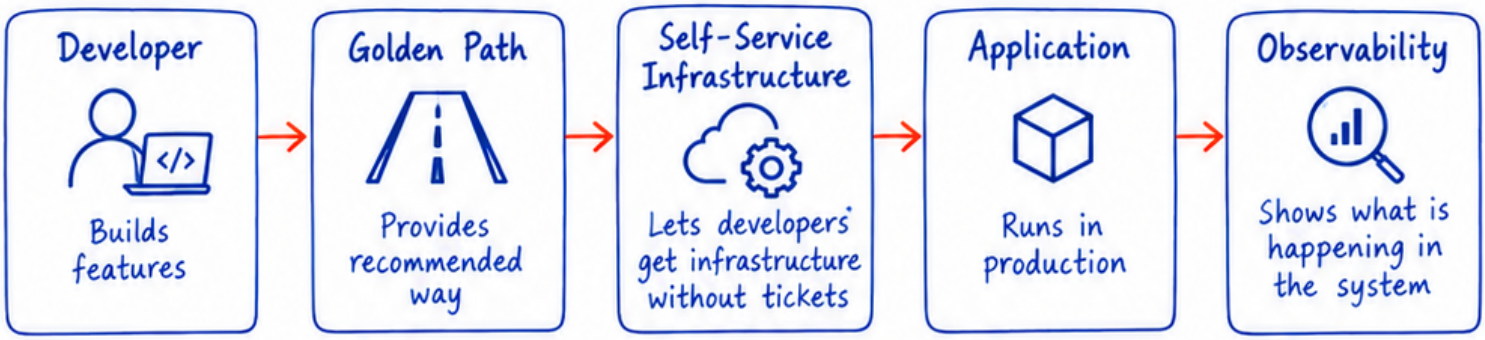
In modern DevOps and platform engineering, Observability, Golden Paths and Self-Service Infrastructure work together to create a smooth, secure and scalable developer experience. They help organizations ship software faster, with higher reliability and better governance.

Platform engineering provides the foundation, and these three pillars ensure that developers can build, deploy and operate applications efficiently without unnecessary friction.

People
Platforms
Progress

2. How They Work Together

A simplified flow from developer to observability



Vyomanant Academy

Bennter insights for a better tomorrow !

3. Why It Matters ?

1. Faster developer onboarding — Get productive quickly
2. Standardized delivery — Consistent and reliable
3. Less manual ticketing — Empower developers
4. Better troubleshooting — Find and fix issues faster
5. Stronger governance — Stay compliant and secure

"A great platform turns complexity into possibility."

Build
Enable
Innovate

4. Quick Reference

Concept	Key Question It Answers
Observability	What is happening in the system?
Golden Path	What is the recommended way to build/deploy?
Self-Service Infrastructure	How can developers get infra without tickets?

Three pillars.
A stronger
Developer Experience.
Together.

People + Platforms + Possibilities | That's Platform Engineering !

Learn
Build
Grow
Together

Observability - Basics

"See Understand Solve"

Understand · Diagnose · Improve

1. Definition

Observability is the ability to understand the internal state of a system using the data it produces.

More Visibility Better Systems

2. Why it matters

1. Detect issues faster — Find problems early
2. Reduce downtime — Keep services running
3. Improve reliability — Build trust and stability
4. Understand user impact — See how issues affect users
5. Speed up troubleshooting — Get to the root cause faster

"Observability turns data into understanding"

Observe Understand Improve

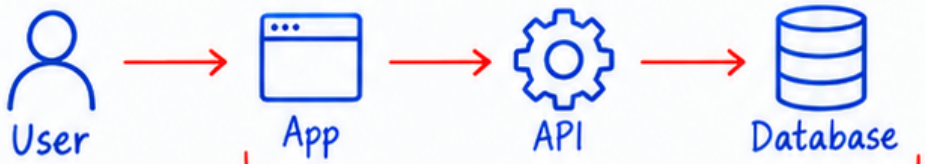
3. Monitoring vs Observability

Vyomanant Academy

Monitoring	VS	Observability
Tells you something is wrong.		Helps you understand why.
"Hey! Something is down!"		"Let's find out what's causing it!"

4. Example

App is slow → Observability helps find whether the issue is with the API, Database, Network, or Infrastructure.



Observability helps you trace, analyze and find the real cause.

Better Visibility | Better Decisions | Brighter Tomorrow

Learn Build Grow Together

Observability - The 3 Pillars

"See Understand Improve"

Metrics + Logs + Traces = Full Visibility

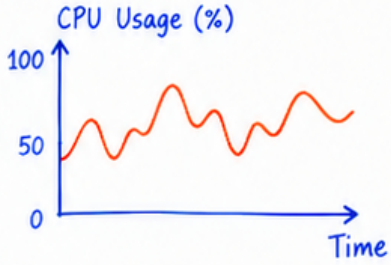
1. Metrics - Numbers Over Time

Metrics
Numbers over time

Metrics are numerical measurements collected over time. They tell us how the system is performing and help us spot trends, anomalies and capacity needs.

Examples:

- CPU usage
- Memory usage
- Requests per second (RPS)
- Latency (response time)



Measure
Monitor
Find Trends

2. Logs - Event Records That Tell What Happened

Logs
Events and messages

Logs are timestamped records of events that tell us what happened in the system. They are useful for understanding failures, user actions and system behavior in detail.

Examples:

- Error messages
- Login events
- Startup and shutdown messages
- Application events (e.g., order created, payment failed)

Find Issues
Understand Context
Debug Faster

```
2024-06-01 10:00:01  INFO  User login successful
2024-06-01 10:01:23  ERROR Payment failed for order #1234
2024-06-01 10:01:24  INFO  Retrying payment...
2024-06-01 10:01:25  INFO  Payment succeeded
```

Vyomanant Academy

3. Traces - Show the Path of a Request

Traces
Request flow across services

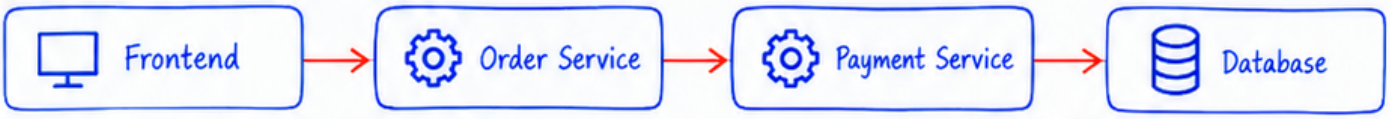
Traces show the complete path of a request as it travels through multiple services. They help identify where time is spent and which service is causing delays or failures.

Examples:

- Follow a user request end-to-end
- See latency at each service
- Identify bottlenecks
- Understand dependencies between services

Visualize the Flow
Find Bottlenecks
Speed Up Debugging

Example: A simple request flow



Example Trace (Approximate Timings)

Frontend	Order Service	Payment Service	Database
100 ms	200 ms	7500 ms	200 ms

Together, metrics + logs + traces give better troubleshooting.

Observe
Correlate
Resolve

Observability in Kubernetes

"Observe Understand Improve"

Metrics, Logs, Traces for Reliable and Scalable Applications

1. Common tools

Prometheus	=	metrics	(collects and stores time-series data)
Grafana	=	dashboards	(visualizes data and creates alerts)
Loki	=	logs	(stores and queries log data)
OpenTelemetry	=	instrumentation	(collects telemetry from applications)
Jaeger / Tempo	=	traces	(distributed tracing for request flow)

Kubernetes Observability Stronger Systems

2. Four Golden Signals

Latency
= response time
How long it takes to serve a request?

Traffic
= requests / load
How much traffic is coming in?

Errors
= failures / 5xx
How many requests are failing?

Saturation
= resource usage
Are our resources (CPU, memory, etc.) close to limits?

These signals give a quick and clear picture of the health of your system.

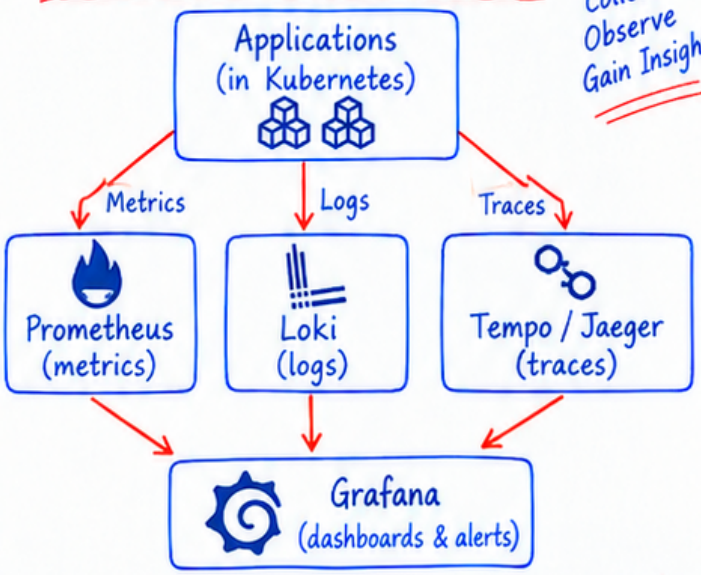
Vyomanant Academy

3. Kubernetes view

What to observe:

- Pods – health, restarts, resource usage
- Nodes – CPU, memory, disk, network
- Containers – logs, resource usage, status
- Deployments – replicas, rollout status
- Services – latency, errors, traffic
- Ingress – request rates, latency, errors
- Databases – performance, connections, errors

Simplified observability architecture



Collect Observe Gain Insights

4. Key Takeaway

Good observability supports alerting, SLOs, and incident response.

Observe Today A More Reliable Tomorrow

Golden Paths - Basics

"Enable Build Scale"

Make the Right Way the Easy Way

1. Definition

A Golden Path is the recommended, supported, and easiest way for developers to perform a common task.

2. Without a Golden Path

Developers choose different repo structures, Dockerfiles, CI pipelines, manifests, and security approaches. This leads to inconsistency, duplication of effort, more troubleshooting and operational overhead.

Different Choices
More Confusion

3. With a Golden Path

Teams get a ready-made path with consistency, security, and best practices. Developers can focus on building features instead of figuring out the plumbing.

More Time for Innovation

4. Main idea

Make the right way the easy way.

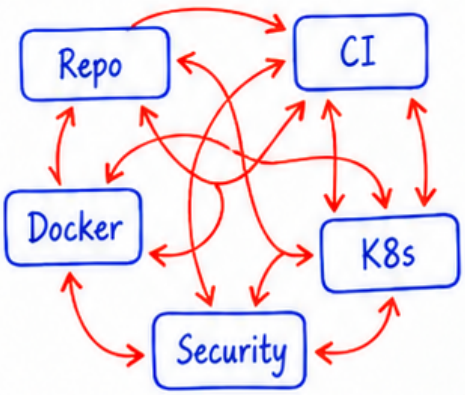
Provide paved roads, not a blank map.

"Good platform teams remove complexity, not freedom."

Build
Enable
Innovate

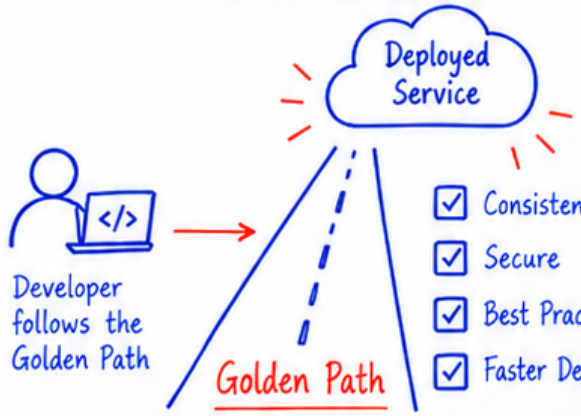
Vyomanant Academy

Without a Golden Path



Different setups
Inconsistent
More work
Higher risk

With a Golden Path



Golden paths reduce cognitive load and speed up delivery.

Make Developers Successful

Less Complexity
More Progress

People
Platforms
Possibilities

Learn
Build
Grow
Together

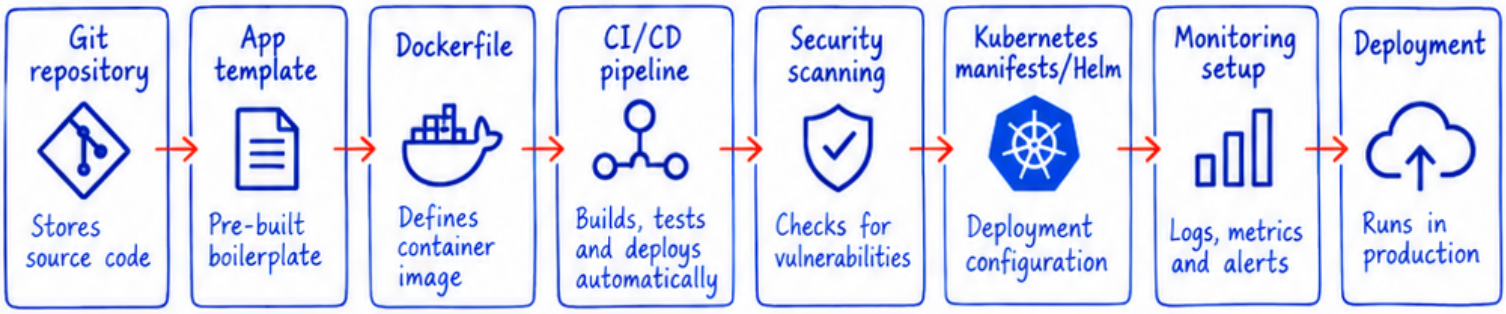
Golden Paths - What They Include

"Paved Roads for Faster Progress"

Standardized, Secure and Ready-to-Use Building Blocks

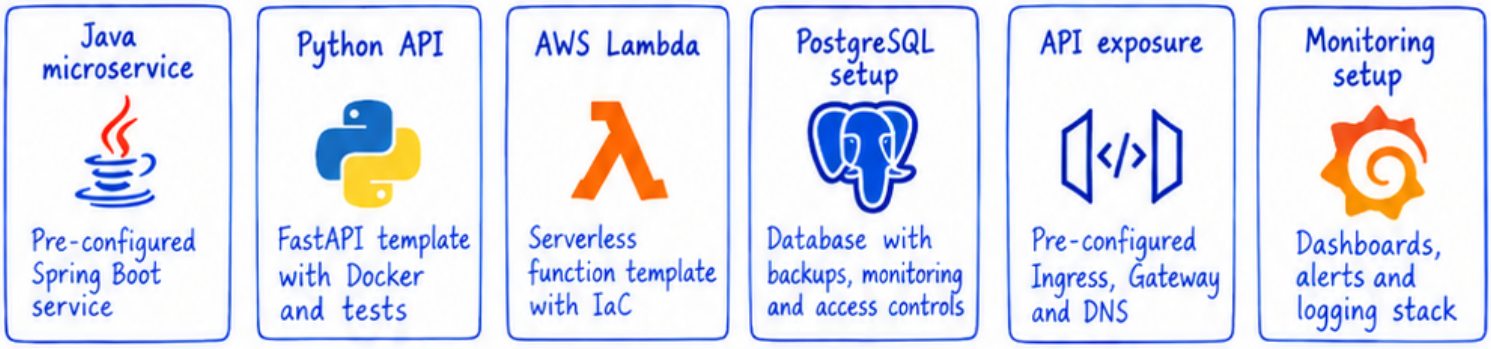
1. Typical components

A golden path provides a pre-defined, best-practice flow with all the essential components developers need to build, deploy and operate an application.



2. Examples of golden paths

Golden paths can be tailored for different types of applications and use cases.



Vyomanant Academy

3. Benefits

1. Consistency → Everyone follows best practices.
2. Faster onboarding → New developers can start building quickly.
3. Easier support → Standardized setups are simpler to maintain.
4. Security-by-default → Security checks and policies are built in.
5. Observability-by-default → Metrics, logs and alerts are enabled from day one.

Clear Paths
Stronger Teams
Brighter Outcomes

4. Important note

A golden path should be the easiest and recommended route, not an unnecessary rigid restriction. It should enable flexibility where needed, while guiding developers towards safe, reliable and best-practice choices.

"Guide Don't Restrict"

Less Friction + More Focus + Better Software = That's Platform Engineering!

Build
Enable
Grow
Together

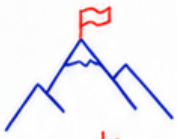
Self-Service Infrastructure - Basics

Empower Developers | Automate Provisioning | Build at Scale

"Enable Build Scale"

1. Definition

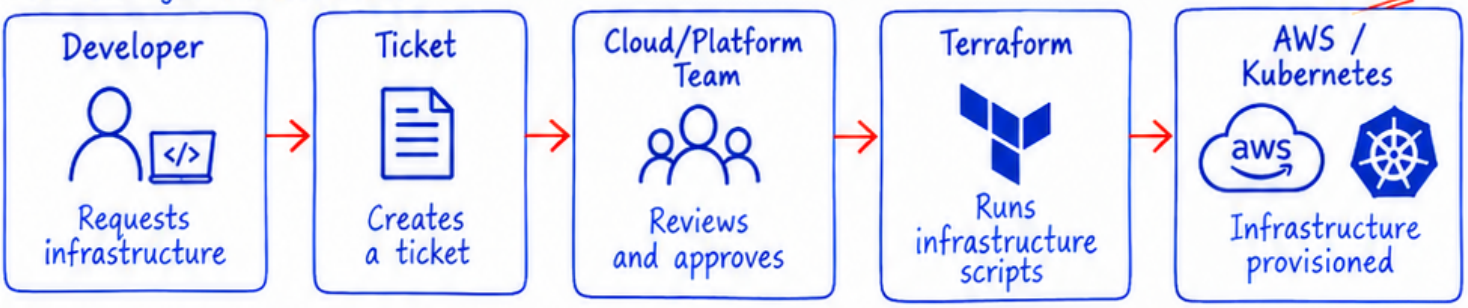
Self-Service Infrastructure allows developers to provision approved infrastructure themselves through automated, governed workflows.



People Platforms Progress

2. Traditional model

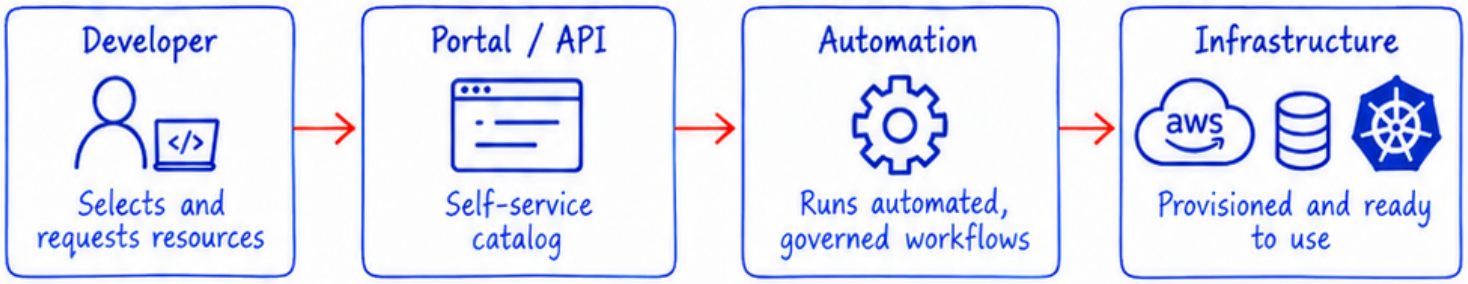
Developers rely on manual processes and multiple teams to get infrastructure.



3. Self-service model

Vyomanant Academy

Developers provision infrastructure themselves through a self-service portal or API.



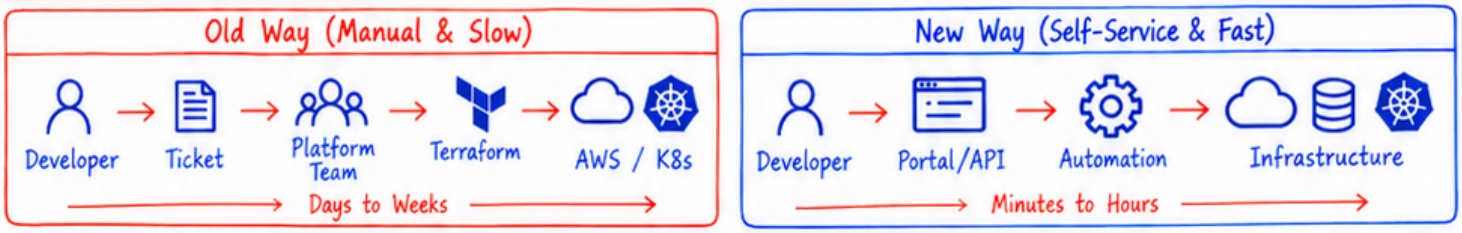
4. Why It Matters ?

1. Faster provisioning — Get infrastructure quickly
2. Less waiting — No more long ticket queues
3. Reduced manual work — Automate repetitive tasks
4. Better developer autonomy — Empower developers to build
5. Standardization — Use approved, consistent templates

"Self-service turns ideas into reality, faster."

Build Enable Innovate

5. Old vs New Flow



People + Platforms + Possibilities | That's Platform Engineering !

Learn Build Grow Together

Self-Service Infrastructure - Workflow & Governance

"Self-Service with Guardrails"

Empower Developers | Automate Provisioning | Govern by Default

1. How it works

Developer Portal
Browse, configure and request infrastructure

→

Platform API
Validates, applies policies and orchestrates

→

Terraform / GitOps
Infrastructure as code with automated workflows

→

AWS / Kubernetes
Provisions and manages resources

Platforms Enable People

2. Common self-service requests

- **Create namespace** — For your application
- **Create S3 bucket** — With security and lifecycle policies
- **Create PostgreSQL DB** — Managed with backups and monitoring
- **Create Redis** — For caching needs
- **Create Kafka topic** — For event streaming
- **Create IAM role** — With least privilege access
- **Create CI/CD pipeline** — Build, test and deploy automation

Vyomanant Academy

3. Built-in controls

- Encryption at rest and in transit
- Mandatory tagging (owner, env, cost center)
- Automated backups and retention
- Approved instance sizes and configurations
- Network policies (VPC, security groups, etc.)
- IAM permissions with least privilege
- Monitoring, logging and alerting by default

"Self-service doesn't mean no control.
It means control by design."

Governance Checklist

- [x] Request validated against policy
- [x] Required tags applied
- [x] Approved instance sizes only
- [x] Network and security rules enforced
- [x] IAM permissions scoped correctly
- [x] Resource created via IaC (GitOps)
- [x] All actions are logged and auditable

Safe ✓ Compliant ✓ Self-Service ✓

4. Platform building blocks

- **Terraform modules** — Reusable, tested infrastructure modules
- **Helm charts** — Standardized deployments for Kubernetes
- **App templates** — Pre-configured application stacks
- **CI/CD templates** — Ready-to-use pipelines (build, test, deploy)
- **Policies** — Composable policy as code (OPA, Sentinel, IAM, etc.)

Automate Standardize Govern Scale

People + Platforms + Possibilities = That's Platform Engineering !

Learn Build Grow Together

www.vyomanant.com academy@vyomanant.com +91 9704286195

How All 3 Concepts Work Together

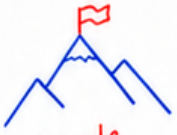
"Enable Build Scale"

From Developer Idea to a Reliable Service in Production

1. Scenario

A developer wants to create a new Payment Service.

Instead of dealing with infrastructure, configs, pipelines and monitoring from scratch, the developer uses the platform capabilities — Golden Path, Self-Service Infrastructure and Observability — to build, deploy and operate the service efficiently.



People
Platforms
Progress

2. Golden Path provides

- **Application template** — Pre-built, best practices
- **Source code repository** — Ready to use repo setup
- **CI/CD pipeline** — Build, test and deploy
- **Docker configuration** — Containerization setup
- **Deployment pattern** — Helm / K8s manifests
- **Monitoring baseline** — Pre-configured dashboards, alerts and SLOs

3. Self-Service Infrastructure provides

- **Namespace** — Isolated environment for the service
- **Database** — Provisioned and ready to use
- **Kafka topic** — Messaging infrastructure
- **Secrets** — Credentials and sensitive configurations
- **Permissions** — RBAC and access control

4. Observability provides

- **Metrics** — Application and infrastructure metrics
- **Logs** — Centralized and searchable logs
- **Traces** — End-to-end request tracing
- **Dashboards** — Real-time visibility
- **Alerts** — Proactive notifications
- **SLOs** — Service level objectives and error budgets

Vyomanant Academy

5. How It All Fits Together

A seamless flow from idea to impact

Developer → Creates a new Payment Service

Internal Developer Portal → Single entry point to templates, tools and services

Golden Path + Self-Service Infrastructure
- Golden Path: App template, CI/CD, Docker, deployment pattern
- \+ Self-Service Infrastructure: Namespace, Database, Kafka, Secrets, Permissions

AWS / Kubernetes → Runs in a scalable, secure and reliable environment

Application → Payment Service Running in production

Observability → Metrics, Logs, Traces, Dashboards, Alerts, SLOs Continuous visibility and insights

Feedback loop for continuous improvement

6. Outcome

Faster Delivery
- ✓ Developers can ship features quickly
- ✓ Less manual work and setup time
- ✓ Focus on business logic

Higher Reliability
- ✓ Best practices built-in
- ✓ Consistent and repeatable setups
- ✓ Proactive monitoring and alerts
- ✓ SLO-driven operations

Better Governance
- ✓ Standardized and compliant
- ✓ Clear ownership and access control
- ✓ Visibility across all services
- ✓ Easier to scale and manage

"Together we build better software!"

People + Platforms + Possibilities | That's Platform Engineering!

Learn
Build
Grow
Together

"Enable Build Scale"

Quick Revision Sheet

Observability, Golden Paths & Self-Service Infrastructure

1. Key Concepts at a Glance

Concept	Key Question	Examples / What it Includes
Observability	What is happening in the system?	Metrics, logs, traces
Golden Path	What is the recommended way to build/deploy?	Templates, CI/CD, standards
Self-Service Infrastructure	How can developers get infra without tickets?	Portal, APIs, automation

Simple Concepts Big Impact

2. Key Benefits

Speed
Ship software faster

Consistency
Standardized and reliable delivery

Autonomy
Empower developers

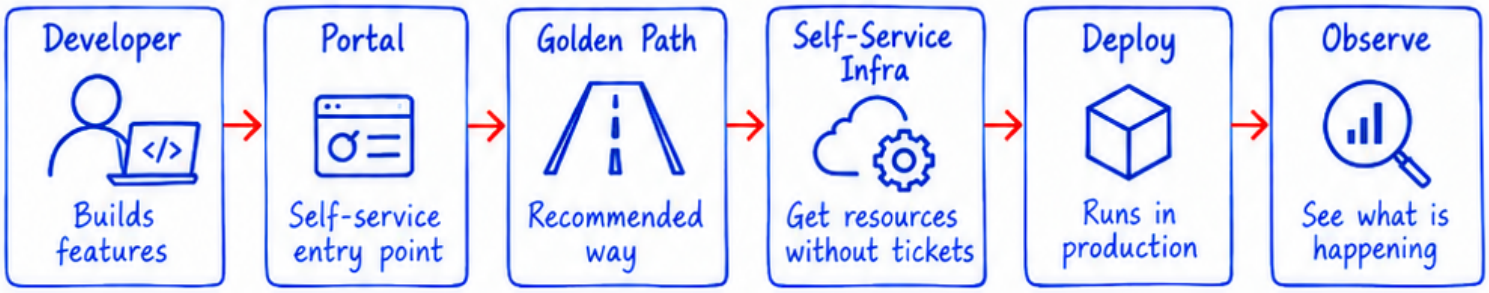
Reliability
More stable and resilient systems

Governance
Stay compliant and in control

Better Platforms Happier Developers

Vyomanant Academy

3. Remember This Flow



4. One-Line Summary

"A strong platform makes software delivery easier, safer, and more scalable."

Learn Build Grow

Same People Bigger Possibilities

People + Platforms + Possibilities ≈ A Brighter Tomorrow!

Learn Build Grow Together